

SmartSure Insurance Management System

Low-Level Design (LLD) Document

Backend: ASP.NET Core 10 Microservices | Frontend: Angular 21

Version 2.0 | March 2026 | Complete & Authoritative

1. System Overview

SmartSure is a full-stack insurance management platform composed of four ASP.NET Core 8 microservices behind an Ocelot API Gateway, with an Angular 21 SPA as the client. Each service owns its own SQL Server 2022 database. Asynchronous inter-service communication uses RabbitMQ (via MassTransit). Authentication is JWT HS256 validated independently in every service.

1.1 Architecture Summary

Component	Technology	Port	Responsibility
Angular SPA	Angular 21 / TypeScript 5.9	4200	User and Admin UI; standalone components, no NgModules
API Gateway	Ocelot + SwaggerForOcelot	5057	Single entry-point: routing, CORS, aggregated Swagger docs
Identity Service	ASP.NET Core 10	5168	Auth, JWT (HS256), OTP, Google OAuth 2.0, BCrypt passwords
Policy Service	ASP.NET Core 10	5014	Policies, payments, IDV calculation, discounts, insurance types
Claims Service	ASP.NET Core 10	5265	Claims lifecycle, document storage on Mega.nz
Admin Service	ASP.NET Core 10	5232	Audit logs, dashboard stats, report generation (PDF/JSON)
RabbitMQ	rabbitmq:3-management (Docker)	5672 / 15672	Async event bus; all services publish/consume via MassTransit
SQL Server	Sql-server:2026-latest	1433	4 isolated databases, one per microservice

1.2 Cross-Cutting Concerns

- Authentication: JWT Bearer (HS256). Token issued by Identity Service; all other services validate using shared Jwt:Key, Jwt:Issuer, Jwt:Audience from appsettings.
- Refresh Tokens: Stored in IMemoryCache in the Identity Service with a 24-hour TTL (not persisted to DB).
- Logging: Serilog request logging via shared SerilogExtensions enabled in Gateway, Claims Service, and Admin Service.
- Error Handling: GlobalExceptionHandlerMiddleware (from SmartSure.Shared.Contracts) maps domain exceptions to standardised HTTP responses.
- Environment Config: DotNetEnv loads .env files; services also consume appsettings.json / appsettings.Development.json and environment variables.
- Repository Pattern: Policy Service uses explicit repository classes (IPolicyRepository, IInsuranceRepository, IDiscountRepository, IPaymentRepository). Claims and Identity use DbContext directly via services.

2. Shared Contracts Library (SmartSure.Shared.Contracts)

A class library referenced by all four microservices. Defines shared DTOs, domain events, exceptions, constants, and middleware to ensure consistency across service boundaries.

2.1 Common Response DTOs

- `ApiResponse<T>`: Generic wrapper with Success (bool), Data (T), Message (string), Errors (List<string>). Static factories: `Ok(data)`, `Fail(message)`.
- `ApiResponse`: Non-generic variant for void operations.
- `PagedResult<T>`: Items (List<T>), TotalCount, Page, PageSize, TotalPages (computed), HasPrevious, HasNext.

2.2 Domain Events (RabbitMQ Messages)

All events are C# record types published and consumed via MassTransit. `ConfigureEndpoints` auto-creates queues based on consumer class names.

Event (C# record)	Key Fields	Publisher → Consumers
UserRegisteredEvent	UserId, Email, FullName, PhoneNumber, RegisteredAt, IsGoogleAuth	Identity → Admin (audit), Policy (user sync)
PolicyActivatedEvent	PolicyId, UserId, TypeId, SubTypeId, ActivatedAt	Policy → Claims (context sync), Admin (audit)
PolicyCancelledEvent	PolicyId, UserId, Reason, CancelledAt	Policy → Admin (audit)
ClaimSubmittedEvent	ClaimId, PolicyId, UserId, Description, SubmittedAt	Claims → Admin (audit)
ClaimStatusChangedEvent	ClaimId, OldStatus, NewStatus, ChangedBy, UserId, Reason	Claims → Admin (audit), Identity (email notification)
ClaimApprovedEvent	ClaimId, AdminId, Notes, ApprovedAt, UserEmail, UserName	Claims → Identity (email notification)
ClaimRejectedEvent	ClaimId, AdminId, Reason, RejectedAt, UserEmail, UserName	Claims → Identity (email notification)

2.3 Exception Hierarchy

Exception Class	HTTP Status	Use Case
SmartSureException	Base	Base class; all domain exceptions derive from this
NotFoundException	404	Entity not found by ID; thrown as <code>NotFoundException("Entity", id)</code>
UnauthorizedException	401	Invalid credentials or missing/invalid token
BusinessRuleException	400	Domain rule violated (e.g. duplicate email, wrong status for transition)
ConflictException	409	Duplicate resource (e.g. filing a claim on a policy already under review)
ValidationException	422	Input validation failure (e.g. invalid OTP, bad password)
HttpServiceException	Proxied	Used by Admin Service when an upstream HTTP proxy call fails

2.4 Constants

- `ClaimStatus`: Static string constants Draft, Submitted, UnderReview, Approved, Rejected, Closed — used across Claims and Admin services to prevent stringly-typed comparisons.

3. API Gateway (Ocelot)

Port: 5057 | **Technology:** Ocelot + SwaggerForOcelot

Single entry-point for the Angular SPA. Handles routing, CORS, and aggregated Swagger UI. JWT validation is NOT performed here — each downstream service validates tokens independently.

3.1 Route Table (ocelot.json)

Upstream Path (Client)	Downstream Path	Service Port	HTTP Methods
/auth/{everything}	/auth/{everything}	Identity 5168	All verbs
/gateway/auth/google/callback	/auth/google/callback	Identity 5168	GET, POST, OPTIONS
/claims/{everything}	/claims/{everything}	Claims 5265	All verbs
/claims	/claims	Claims 5265	GET, POST, OPTIONS
/admin/{everything}	/admin/{everything}	Admin 5232	All verbs
/ {everything} (catch-all)	/ {everything}	Policy 5014	All verbs

3.2 CORS Policy

Policy name: "AllowAngularApp". Allows credentials and all HTTP methods. Permitted origins:

- http://localhost:4200
- https://localhost:4200
- http://localhost:5057
- https://localhost:9000

HTTPS redirect is intentionally disabled to prevent CORS preflight redirect errors during local development.

3.3 Swagger Aggregation

SwaggerForOcelot aggregates Swagger endpoints from all downstream services and presents a unified API explorer at /swagger on the gateway.

4. Identity Service

Port: 5168 | **Database:** IdentityConnDb | **Route prefix:** /auth

4.1 Data Models

User (Primary Entity)

Property	Type	Notes
UserId	Guid (PK)	Primary key, auto-generated
FullName	string	Required; user's full name
Email	string	Required; [EmailAddress] validated; unique
PhoneNumber	string	Required
Address	string?	Optional
IsEmailVerified	bool	Default false; set true after OTP confirmation
IsGoogleAuth	bool	True if registered via Google OAuth — no password row stored
CreatedAt	DateTime	UTC timestamp of account creation
Password	Navigation	One-to-one to Password table (separate for security)
UserRoles	Navigation	Many-to-many through UserRole join table
ExternalLogins	Navigation	Collection of linked OAuth provider records
OtpRecords	Navigation	Collection of OTP attempts for this user

Supporting Models

- **Password:** PassId (PK), UserId (FK), PasswordHash (BCrypt). Stored in a separate table to avoid hash exposure in User queries.
- **Role:** RoleId (PK), RoleName. Seeded at startup with Admin and User roles.
- **UserRole:** Id (PK), UserId (FK), RoleId (FK). Join entity for the many-to-many User-to-Role relationship.
- **OtpRecord:** Id, UserId, Email, Otp (6-digit string), ExpirationTime (10 min), Attempts (int, max 5). Used for registration OTP and password-reset OTP. Record deleted on successful verification.
- **ExternalLogin:** Id, UserId, Provider (Google), ProviderKey (Google sub ID), Email, CreatedAt. One record per OAuth provider linked.
- **AdminSeeder:** Seeds the default admin account and Role records on startup.

4.2 API Endpoints

Verb	Path	Auth	Description
GET	/auth/google	None	Redirects browser to Google OAuth consent page
GET	/auth/google/callback	None	Exchanges Google code for JWT; redirects to Angular with token
POST	/auth/register	None	Create user account; sends OTP for email verification
POST	/auth/verify-register-otp	None	Verifies OTP; sets IsEmailVerified=true; publishes UserRegisteredEvent
POST	/auth/login	None	Validates BCrypt hash; returns {token, role, refreshToken}
POST	/auth/forgot-password	None	Generates 6-digit OTP and emails it to the registered address

POST	/auth/reset-password	None	Validates OTP; hashes and persists new password
GET	/auth/me	[Authorize]	Returns authenticated user's profile via UserId from JWT claim
PUT	/auth/me	[Authorize]	Updates FullName, PhoneNumber, Address
PUT	/auth/change-password	[Authorize]	Validates old password; hashes and stores new password
POST	/auth/refresh	None	Exchanges refresh token (IMemoryCache, 24hr TTL) for new JWT pair
GET	/auth/users	[Admin]	Lists all users with their roles
PUT	/auth/users/{userId}/roles	[Admin]	Assigns a role (by RoleId) to a user
DELETE	/auth/users/{userId}	[Admin]	Permanently deletes a user and all related data

4.3 Service Layer

AuthService

Registration: hashes password with BCrypt (via PasswordHasher helper), creates User record, creates Password record, triggers OtpService for email delivery. After OTP verification, publishes UserRegisteredEvent.

Login: validates BCrypt hash, assembles JWT claims (NameIdentifier=UserId, Email, Role), calls TokenService.BuildToken, generates refresh token stored in IMemoryCache with a 24-hour TTL.

Profile: GetProfile/UpdateProfile by UserId from JWT NameIdentifier claim. ChangePassword validates old hash before storing new one.

TokenService (Singleton)

BuildToken: generates HS256 signed JWTs with ClaimTypes.NameIdentifier (UserId), ClaimTypes.Email, ClaimTypes.Role. Expiry configured via appsettings. GenerateRefreshToken returns a cryptographically random opaque string.

OtpService

Generates a random 6-digit OTP, persists an OtpRecord with 10-minute expiry, calls EmailService. Verification checks expiry and Attempts counter (max 5 = lockout). On success, deletes the OtpRecord.

GoogleAuthService

Builds Google OAuth 2.0 authorization URL. On callback: exchanges code for Google ID token, decodes it to extract sub (ProviderKey), email, and name. Upserts User and ExternalLogin records, then issues a SmartSure JWT.

EmailService

Sends SMTP emails (OTP codes, welcome messages, claim notifications). Config from appsettings: SmtpHost, SmtpPort, SmtpUser, SmtpPass.

UserRepository / UserService (Admin Operations)

Repository pattern over IdentityDbContext. UserService provides GetUsers (all, with roles), AssignRole, DeleteUser for admin operations.

MassTransit Consumer

- ClaimStatusChangedConsumer: Receives ClaimStatusChangedEvent; sends email notifications to users when their claim status changes (Approved or Rejected).

5. Policy Service

Port: 5014 | **Database:** PolicyConnDb | **Route prefixes:** /policies, /insurance-types, /insurance-subtypes, /discounts, /payments

5.1 Data Models

Policy (Core Entity)

Property	Type	Notes
PolicyId	Guid (PK)	Primary key, auto-generated
UserId	Guid	References Identity user (no cross-service FK constraint)
SubtypeId	Guid (FK)	References InsuranceSubtype
StartDate / EndDate	DateTime (UTC)	Coverage period; EndDate = StartDate + Duration months
PremiumAmount	decimal(18,2)	Final premium after IDV calculation and discount application
InsuredDeclaredValue	decimal(18,2)	System-calculated IDV: depreciation-based (vehicle) or rebuild-cost (home)
Status	string(50)	Pending Active Cancelled Expired
NomineeName	string?(100)	Optional nominee name on the policy
NomineeRelation	string?(50)	Optional nominee relationship on the policy
CreatedAt	DateTime (UTC)	Auto-set at creation
PolicyDetail	Navigation	One-to-one: TermsAndConditions, Inclusions, Exclusions
HomeDetail	Navigation	Nullable; only for home insurance policies
VehicleDetail	Navigation	Nullable; only for vehicle insurance policies
Payments	Navigation	One-to-many collection of Payment records

Supporting Models

- InsuranceType: TypeId (PK), Name, Description, Subtypes (nav). Top-level categories e.g. Vehicle, Home.
- InsuranceSubtype: SubtypeId (PK), TypeId (FK), Name, Description, BasePremium (decimal). E.g. Comprehensive, Third-Party.
- PolicyDetail: DocumentId (PK), PolicyId (FK), TermsAndConditions, Inclusions, Exclusions (strings).
- VehicleDetail: VehicleDetailId (PK), PolicyId (FK), RegistrationNumber, Make, Model, ManufactureYear, EstimatedValue, ChassisNumber, EngineNumber.
- HomeDetail: HomeDetailId (PK), PolicyId (FK), Address, PropertyType, YearBuilt, EstimatedValue, SecurityFeatures.
- Payment: PaymentId (PK), PolicyId (FK), Amount, PaymentDate, Status (Success/Failed/Pending), PaymentMethod, TransactionReference.
- Discount: DiscountId (PK), Code (unique, stored UPPER), Description, Percentage (decimal 5,2), MaxDiscountAmount, IsFirstTimeOnly (bool), IsActive, ValidFrom, ValidUntil.

5.2 API Endpoints

Verb	Path	Auth	Description
GET	/policies	[Authorize]	Get all policies for the authenticated user
GET	/policies/all	[Admin]	Get all policies across all users
GET	/policies/{policyId}	[Authorize]	Get a specific policy by ID

POST	/policies/quote	[Authorize]	Calculate IDV + premium (no policy created); returns PolicyQuotedDTO with Breakdown string
POST	/policies	[Authorize]	Create policy in Pending status; applies coupon + first-time discounts
PUT	/policies/{policyId}/activate	[Authorize]	Transitions Pending → Active; publishes PolicyActivatedEvent
PUT	/policies/{policyId}/cancel	[Authorize]	Transitions to Cancelled; publishes PolicyCancelledEvent
GET	/policies/{policyId}/details	[Authorize]	Returns T&C, Inclusions, Exclusions for a policy
POST	/policies/{policyId}/details	[Authorize]	Saves a new PolicyDetail document
PUT	/policies/{policyId}/details	[Authorize]	Updates an existing PolicyDetail document
GET	/home-details/{policyId}	[Authorize]	Returns HomeDetail for a policy
GET	/vehicle-details/{policyId}	[Authorize]	Returns VehicleDetail for a policy
GET	/insurance-types	None	List all insurance types (public)
POST/PUT/DELETE	/insurance-types/{id}	[Admin]	Create, update or delete an insurance type
GET	/insurance-types/{typeId}/subtypes	None	Get subtypes for a specific type
GET	/insurance-subtypes	None	List all subtypes (public)
POST/PUT/DELETE	/insurance-subtypes/{id}	[Admin]	Create, update or delete a subtype
GET	/discounts	None	List all active discount coupons (public)
POST	/discounts/calculate	[Authorize]	Calculate discount for a given premium and coupon code
POST/PUT/DELETE	/discounts/{id}	[Admin]	Create, update or delete a discount coupon
GET	/policies/{policyId}/payments	[Authorize]	List payments for a policy
POST	/policies/{policyId}/payments	[Authorize]	Record a payment (Amount, Status, PaymentMethod, TransactionReference)

5.3 IDV and Premium Calculation Logic

The following business rules are implemented in PolicyMgmtService and are central to the quoting and policy creation flows:

Parameter	Rule / Value
Vehicle IDV Formula	$IDV = EstimatedValue \times (1 - DepreciationRate)$; minimum ₹10,000
Depreciation ≤6 months	5% (age = 0)
Depreciation 6–12 months	15% (age = 1)
Depreciation 1–2 years	20% (age = 2)
Depreciation 2–3 years	30% (age = 3)
Depreciation 3–4 years	40% (age = 4)
Depreciation 4–5 years	50% (age = 5)

Depreciation >5 years	60% (age > 5)
Home Insurance Value	$\text{EstimatedValue} \times (1 - \text{AgeDepreciation}) \times \text{PropertyMultiplier}$; minimum ₹50,000
Age Depreciation	1% per year since YearBuilt; capped at 40%
Property Multiplier	Apartment = 0.90 House = 1.00 Villa = 1.10
Premium Formula	$\text{Premium} = (\text{BasePremium} \times \text{Duration}/12) + (\text{IDV} \times \text{RiskRate} \times \text{Duration}/12)$
Risk Rate – Vehicle	2.5% per year of IDV
Risk Rate – Home	0.15% per year of IDV
Discount Cap	Total discount capped at 30%; MaxDiscountAmount per coupon also respected
First-time Buyer	Automatic 10% discount for users with zero existing policies

5.4 Service Layer

PolicyMgmtService

- **CalculateQuoteAsync**: Looks up subtype and type, determines vehicle/home by type name, calculates IDV, computes premium, returns PolicyQuoteDTO with a human-readable Breakdown string. No database writes.
- **CreatePolicyAsync**: Calculates IDV + premium, calls DiscountService.CalculateDiscountAsync (applies first-time 10% and coupon), creates Policy in Pending status, persists HomeDetail or VehicleDetail as appropriate.
- **ActivatePolicyAsync**: Transitions Pending → Active; publishes PolicyActivatedEvent via RabbitMQ. Guard: if already Active, returns immediately.
- **CancelPolicyAsync**: Delegates to IPolicyRepository.CancelAsync; publishes PolicyCancelledEvent.
- **GetPolicyDetailsAsync** / **SavePolicyDetailsAsync**: Read/write PolicyDetail records.
- **GetHomeDetailAsync** / **GetVehicleDetailAsync**: Read asset-specific detail records.

DiscountService

- **CalculateDiscountAsync**: Checks IsFirstTimeOnly eligibility by counting user policies (0 = first-time → 10% auto discount), applies coupon Code match, validates active + not expired, respects MaxDiscountAmount cap, caps total at 30%.
- **CRUD**: GetAll, GetById, Create, Update, Delete discount coupon records.

InsuranceService

Provides CRUD for InsuranceType and InsuranceSubtype entities including seeding/admin management.

PaymentService

Persists Payment records linked to policies. GetPaymentsForPolicyAsync, AddPaymentAsync.

Repositories

- **IPolicyRepository** / **PolicyRepository**: GetByIdAsync, GetAllAsync, GetByIdAsync, AddAsync, UpdateAsync, CancelAsync, GetDetailByPolicyIdAsync, AddOrUpdateDetailAsync, AddOrUpdateHomeDetailAsync, AddOrUpdateVehicleDetailAsync, GetHomeDetailByPolicyIdAsync, GetVehicleDetailByPolicyIdAsync, GetPremiumAmountAsync, SaveChangesAsync.
- **IInsuranceRepository** / **InsuranceRepository**: GetAllTypesAsync, GetTypeByIdAsync, GetAllSubtypesAsync, GetSubtypeByIdAsync, GetSubtypesByTypeIdAsync, and CRUD operations.
- **IDiscountRepository** / **DiscountRepository**: GetByCodeAsync used for coupon lookup.
- **IPaymentRepository** / **PaymentRepository**: GetByPolicyIdAsync, AddAsync.

MassTransit Consumer

- **UserRegisteredConsumer**: Receives UserRegisteredEvent from Identity Service to sync user data context in the Policy database if needed.

6. Claims Service

Port: 5265 | **Database:** ClaimsConnDb | **Route prefix:** /claims

6.1 Data Models

Claim (Core Entity)

- ClaimId: Guid (PK)
- PolicyId: Guid — references Policy Service (no cross-service FK constraint)
- UserId: Guid — references Identity Service user
- Description: string(500) — required claim description
- Status: string(50) — see status lifecycle table below
- ClaimAmount: decimal(18,2) — amount claimed by user
- ApprovedAmount: decimal? — nullable; set by admin on approval
- RejectionReason: string? — nullable; set by admin on rejection
- ClaimType: string? — e.g. Accident, Theft, Natural Disaster
- IsCompletelyDamaged: bool? — nullable; indicates total loss for vehicle claims
- CreatedAt / UpdatedAt: DateTime (UTC) — audit timestamps
- Documents: Navigation — collection of ClaimDocument records
- StatusHistory: Navigation — collection of ClaimStatusHistory audit trail records

Supporting Models

- ClaimDocument: DocumentId (PK), ClaimId (FK), FileName(255), FileUrl(1000 — Mega.nz URL), ContentType, FileSize (long), UploadedAt.
- ClaimStatusHistory: Id (PK), ClaimId (FK), OldStatus, NewStatus, Notes(500), ChangedBy (UserId string), ChangedAt. Full audit trail of every status transition.

6.2 Claim Status Lifecycle

Status	Triggered By	Notes
Draft	User creates claim	Starting state; edits allowed; no event published yet
Submitted	User submits	ClaimSubmittedEvent + ClaimStatusChangedEvent published; no edits allowed
UnderReview	Admin sets review	Admin sets via PUT /admin/claims/{id}/review; audit log written
Approved	Admin approves	ApprovedAmount set; ClaimStatusChangedEvent published; email sent via Identity
Rejected	Admin rejects	RejectionReason set; ClaimStatusChangedEvent published; email sent via Identity
Closed	System/admin	Terminal state for claims resolved or expired

6.3 Conflict Guard

CreateClaimAsync rejects with ConflictException if a claim already exists for the same PolicyId with status Draft, Submitted, or UnderReview. This prevents duplicate active claims per policy.

6.4 API Endpoints

Verb	Path	Auth	Description
GET	/claims	[Authorize]	Get all claims for authenticated user; ordered by CreatedAt desc
GET	/claims/all	[Admin]	Get all claims across all users

GET	/claims/{claimId}	[Authorize]	Get a specific claim with Documents included
POST	/claims	[Authorize]	Create claim in Draft status; rejects if active claim exists for the policy
PUT	/claims/{claimId}	[Authorize]	Update description/amount — only allowed in Draft status
PUT	/claims/{claimId}/submit	[Authorize]	Draft → Submitted; writes StatusHistory; publishes ClaimSubmittedEvent + ClaimStatusChangedEvent
GET	/claims/{claimId}/history	[Authorize]	Returns full ClaimStatusHistory audit trail ordered desc
PUT	/claims/{claimId}/approve	[Admin]	Sets ApprovedAmount; transitions to Approved; publishes ClaimStatusChangedEvent
PUT	/claims/{claimId}/reject	[Admin]	Sets RejectionReason; transitions to Rejected; publishes ClaimStatusChangedEvent
GET	/claims/by-policy/{policyId}	[Authorize]	Get all claims for a specific policy
GET	/claims/{claimId}/documents	[Authorize]	List all documents attached to a claim
POST	/claims/{claimId}/documents	[Authorize]	Upload via multipart/form-data → uploaded to Mega.nz; URL saved in ClaimDocument
DELETE	/claims/{claimId}/documents/{docId}	[Authorize]	Delete a document from a claim

6.5 Service Layer

ClaimService

- **CreateClaimAsync:** Checks for active claim conflict. Creates Claim in Draft status. Writes initial ClaimStatusHistory entry ('Claim created').
- **UpdateClaimAsync:** Only allowed when Status = Draft; throws BusinessException otherwise. Updates Description and ClaimAmount.
- **SubmitClaimAsync:** Validates Draft status. Transitions to Submitted. Writes StatusHistory. Publishes ClaimSubmittedEvent AND ClaimStatusChangedEvent.
- **ApproveClaimAsync:** Sets ApprovedAmount and Status=Approved. Writes StatusHistory. Publishes ClaimStatusChangedEvent.
- **RejectClaimAsync:** Sets RejectionReason and Status=Rejected. Writes StatusHistory. Publishes ClaimStatusChangedEvent.
- **TransitionStatusAsync:** Generic status-change method used internally and by Admin Service proxy calls. Writes StatusHistory and publishes ClaimStatusChangedEvent.
- **GetClaimHistoryAsync:** Returns ClaimStatusHistory ordered by ChangedAt descending.

DocumentService

- **AddDocumentAsync:** Validates file, uploads to Mega.nz via Mega.nz SDK, saves ClaimDocument record with returned URL.
- **GetDocumentsAsync / DeleteDocumentAsync:** Retrieval and deletion of ClaimDocument records.

MassTransit Consumers

- **PolicyActivatedConsumer:** Receives PolicyActivatedEvent; stores policy context (PolicyId, UserId, TypeId, SubTypeId) in Claims DB for cross-service reference during claim submission.
- **ClaimApprovedConsumer / ClaimRejectedConsumer:** Receive approval/rejection events; can trigger additional post-processing actions.

7. Admin Service

Port: 5232 | **Database:** AdminConnDb | **Route prefix:** /admin

The Admin Service is an orchestrator, not a data owner. It proxies calls to other services via the API Gateway (forwarding the JWT Bearer token) and populates its own AdminDb by consuming RabbitMQ events.

7.1 Data Models

- **AuditLog:** Id (PK), Action(100), EntityType(100), EntityId(Guid?), ActorId(Guid?), Details(500), Timestamp. Records every significant admin action.
- **Report:** ReportId (PK), Title(200), ReportType (PolicySummary | ClaimsSummary | Revenue | UserActivity), Format (PDF | CSV), Content (JSON or file path), GeneratedBy (admin UserId), DateRangeStart, DateRangeEnd, CreatedAt.

7.2 API Endpoints

Verb	Path	Auth	Description
GET	/admin/claims	[Admin]	Proxy → GET /claims/all on Claims Service via gateway
GET	/admin/claims/statistics	[Admin]	Aggregates total/pending/underReview/approved/rejected counts
PUT	/admin/claims/{claimId}/review	[Admin]	Calls TransitionStatusAsync → UnderReview; writes AuditLog
PUT	/admin/claims/{claimId}/approve	[Admin]	Proxy approve to Claims Service; writes AuditLog
PUT	/admin/claims/{claimId}/reject	[Admin]	Proxy reject to Claims Service; writes AuditLog
GET	/admin/policies	[Admin]	Proxy → GET /policies/all on Policy Service
GET	/admin/users	[Admin]	Proxy → GET /auth/users on Identity Service
GET	/admin/users/{userId}	[Admin]	Proxy → GET /auth/users/{userId} on Identity Service
DELETE	/admin/users/{userId}	[Admin]	Proxy → DELETE /auth/users/{userId} on Identity Service
GET	/admin/audit-logs	[Admin]	Paginated audit logs; query params: page (default 1), pageSize (default 20, max 100)
GET	/admin/dashboard	[Admin]	Returns totalUsers, totalPolicies, totalClaims, pendingClaims, totalRevenue
GET	/admin/reports	[Admin]	Lists all previously generated reports
POST	/admin/reports	[Admin]	Generates and persists a JSON report record
POST	/admin/reports/pdf	[Admin]	Generates and streams a binary PDF report file download

7.3 Service Layer

AuditService

- **LogAsync(action, entityType, entityId, actorId, details):** Writes AuditLog record with UTC timestamp.
- **GetAuditLogsAsync(page, pageSize):** Returns paginated logs ordered by Timestamp descending.
- **GetTotalAuditLogsCountAsync():** For pagination metadata.

ReportService

- **GenerateReportAsync:** Fetches data from Policy and Claims services via the gateway (with JWT forwarded), aggregates into a Report record, persists to AdminDb.

- Report types: PolicySummary, ClaimsSummary, Revenue, UserActivity.

PdfGeneratorService

- GeneratePdfReportAsync: Converts aggregated report data into a binary PDF byte array, streamed as a file download from POST /admin/reports/pdf.

MassTransit Consumers (5 total)

- UserRegisteredConsumer: Receives UserRegisteredEvent; writes audit log.
- PolicyActivatedConsumer: Receives PolicyActivatedEvent; writes audit log.
- PolicyCancelledConsumer: Receives PolicyCancelledEvent; writes audit log.
- ClaimSubmittedConsumer: Receives ClaimSubmittedEvent; writes audit log.
- ClaimStatusChangedConsumer: Receives ClaimStatusChangedEvent; writes audit log.

8. Frontend (Angular 21 SPA)

A standalone Angular 21 application (no NgModules). All components use standalone: true. Angular Router handles client-side navigation. Functional HTTP interceptors inject JWT. Functional route guards enforce access control. All API calls target the gateway at `http://localhost:5057` (configured in `environment.ts`).

8.1 Application Bootstrap (`app.config.ts`)

Configured via `ApplicationConfig` with three providers:

- `provideBrowserGlobalErrorListeners()` — global browser error catching
- `provideRouter(routes)` — client-side routing
- `provideHttpClient(withInterceptors([authInterceptor]))` — HTTP with JWT interceptor

Note: `responseInterceptor` is defined in `response.interceptor.ts` but is not yet wired into the HTTP client provider. It is available for future global response transformation or error mapping.

8.2 Route Table (`app.routes.ts`)

Route Path	Component	Guard / Notes
/	Home	Public — landing page with insurance product showcase
/about-us	AboutUs	Public
/contact-us	ContactUs	Public
/insurance	Insurance	Public — browse insurance types and subtypes with pricing
/login	Login	Public — email/password + Google OAuth link
/register	Register	Public — name, email, phone, password form
/verify-otp	VerifyOtp	Public — OTP entry after registration
/forgot-password	ForgotPassword	Public — two-step: request OTP, then OTP + new password
/auth/google/callback	GoogleCallback	Public — extracts JWT from URL param after Google OAuth redirect
/user/dashboard	UserDashboard	userGuard — User or Admin role
/user-dashboard	(redirect)	Redirects to /user/dashboard (legacy alias)
/admin/dashboard	AdminDashboard	adminGuard — Admin role only

8.3 Route Guards

- `authGuard`: Checks if JWT token exists in `localStorage`. If missing, redirects to `/login` with `returnUrl` query param.
- `userGuard`: Checks token exists, then validates role is 'User' or 'Admin' (both allowed). No token → `/login`. Unknown role → `/login`.
- `adminGuard`: Checks token, then strictly validates role is 'Admin'. No token → `/login`. 'User' role → `/user/dashboard`. Unknown role → `/login`.

8.4 HTTP Interceptors

`authInterceptor` (Functional)

Attaches JWT Bearer token to all outgoing HTTP requests. Injects `AuthService.getToken()`. If a token exists AND the request URL does not contain `/auth/login` or `/auth/register`, clones the request and adds `Authorization: Bearer {token}` header. Login and register requests are explicitly excluded to avoid sending stale tokens.

8.5 Core Services

ApiService (Base HTTP Wrapper)

All feature services inherit HTTP calls via ApiService. Base URL from environment.apiUrl. Methods: get<T>, post<T>, put<T>, patch<T>, delete<T>, postFormData<T>. The postFormData method omits Content-Type so the browser sets the correct multipart/form-data boundary automatically.

AuthService

- Storage keys: smartsure_auth_token (JWT), smartsure_auth_role (role string)
- BehaviorSubject authStatus\$: Emits true on login, false on logout. Used by Navbar to show/hide links reactively.
- isAdmin() / isUser(): Case-insensitive role comparison.
- login(credentials): POST auth/login → stores token and role in localStorage, emits true on authStatus\$.
- register(userData): POST auth/register → returns {message}.
- verifyOtp(data): POST auth/verify-register-otp.
- forgotPassword / resetPassword: POST auth/forgot-password and POST auth/reset-password.
- logout(): Removes token and role from localStorage, emits false on authStatus\$.
- getProfile() / updateProfile(): GET and PUT auth/me.
- changePassword(): PUT auth/change-password.
- getGoogleLoginUrl(): Returns \${environment.apiUrl}/auth/google (full gateway URL).
- getAllUsers() / assignRole(userId, roleId) / deleteUser(userId): Admin-only operations.

PolicyService

- getInsuranceTypes() / getInsuranceSubtypes(): GET insurance-types, GET insurance-subtypes.
- calculateQuote(data): POST policies/quote → returns IDV + premium estimate + Breakdown string.
- buyPolicy(data): POST policies → creates policy in Pending status.
- getUserPolicies() / getPolicyById(): GET policies, GET policies/{policyId}.
- activatePolicy(policyId): PUT policies/{policyId}/activate.
- cancelPolicy(policyId): PUT policies/{policyId}/cancel.
- processPayment(policyId, data): POST policies/{policyId}/payments.
- calculateDiscount(premium, coupon): POST discounts/calculate.

ClaimService

- submitClaim(claimData): POST claims → creates claim in Draft status.
- updateClaim(claimId, data): PUT claims/{claimId}.
- submitClaimToReview(claimId): PUT claims/{claimId}/submit → transitions Draft to Submitted.
- getUserClaims() / getClaimDetails(): GET claims, GET claims/{claimId}.
- uploadDocument(claimId, formData): POST claims/{claimId}/documents via postFormData (multipart/form-data).
- getClaimHistory(claimId): GET claims/{claimId}/history.

AdminService

- getAllUsers(): GET auth/users (direct to Identity via gateway).
- getAllClaims(): GET claims/all (direct to Claims Service via gateway).
- getClaimStatistics(): GET admin/claims/statistics.
- approveClaim(claimId, amount, notes): PUT admin/claims/{claimId}/approve.
- rejectClaim(claimId, reason): PUT admin/claims/{claimId}/reject.
- getAllPolicies(): GET policies/all (direct to Policy Service via gateway).
- getAuditLogs(page, pageSize): GET admin/audit-logs?page=N&pageSize=N.
- generateReport(requestData): POST admin/reports.

8.6 Pages and Components

- Home: Public landing page with insurance product showcase and CTAs.

- Login: Email/password login form + Google OAuth link. On success stores token and role, navigates to role-based dashboard.
- Register: Registration form (name, email, phone, password). On success navigates to /verify-otp.
- VerifyOtp: OTP input form for email verification after registration.
- ForgotPassword: Two-step flow — request OTP by email, then enter OTP + new password to reset.
- GoogleCallback: Extracts token from URL query param after Google OAuth redirect. Stores via AuthService, navigates to dashboard.
- UserDashboard: Main user interface — view/buy policies, submit/track claims, upload documents, view payments, update profile.
- AdminDashboard: Main admin interface — manage users/policies/claims, approve/reject claims, manage insurance types and discounts, generate reports, view audit logs.
- Insurance: Public listing of insurance types and subtypes with pricing.
- Navbar (shared component): Subscribes to authStatus\$ BehaviorSubject to show/hide login/logout/dashboard links reactively.
- InsuranceCard (shared component): Reusable card displaying insurance product name, description, and base premium.

9. Key Data Flows

9.1 User Registration and OTP Verification

- User fills Register form → Angular calls POST /auth/register.
- Identity Service: creates User (IsEmailVerified=false), hashes password with BCrypt, stores Password record, generates 6-digit OTP in OtpRecord (10-minute expiry), sends OTP email.
- Angular navigates to /verify-otp. User enters OTP code.
- Angular calls POST /auth/verify-register-otp with {email, otp}.
- Identity Service: validates OTP (checks expiry and attempts count ≤5), sets IsEmailVerified=true, deletes OtpRecord.
- Identity Service publishes UserRegisteredEvent via RabbitMQ → consumed by Admin Service (audit log) and Policy Service (user sync).

9.2 Login and JWT Token Flow

- Angular calls POST /auth/login with {email, password}.
- Identity Service: validates BCrypt hash, assembles JWT claims (NameIdentifier=UserId, Email, Role), signs with HS256, generates refresh token (stored in IMemoryCache, 24-hour TTL).
- Response: {token, role, refreshToken}. AuthService stores token and role in localStorage, emits true on authStatus\$.
- authInterceptor attaches Bearer token to all subsequent requests automatically (excluding login/register).
- Guards read role from localStorage to enforce route-level access control.

9.3 Google OAuth Flow

- User clicks Google login → Angular calls getGoogleLoginUrl() → redirects browser to /auth/google.
- Identity Service builds Google OAuth 2.0 authorization URL and redirects browser to Google.
- Google redirects back to /gateway/auth/google/callback (ocelot routes to Identity /auth/google/callback).
- GoogleAuthService exchanges code for Google ID token, extracts sub/email/name, upserts User and ExternalLogin records.
- Identity Service redirects Angular to /auth/google/callback?token={jwt}.
- GoogleCallback component extracts token from URL, stores via AuthService, navigates to dashboard.

9.4 Policy Purchase Flow

- User selects InsuranceType and InsuranceSubtype. Enters asset details (vehicle or home).

- Angular calls POST /policies/quote → Policy Service returns IDV, premium, and Breakdown string.
- User optionally enters coupon code → POST /discounts/calculate returns discounted premium.
- User confirms → POST /policies creates Policy in Pending status (discount applied inside CreatePolicyAsync).
- Angular calls POST /policies/{policyId}/payments to record payment details.
- Angular calls PUT /policies/{policyId}/activate → status transitions to Active. PolicyActivatedEvent published via RabbitMQ.
- PolicyActivatedEvent consumed by: Claims Service (stores policy context in Claims DB), Admin Service (writes audit log).

9.5 Claim Lifecycle

- User selects a policy and fills claim form → POST /claims creates Claim in Draft status (conflict guard prevents duplicate active claims per policy).
- User uploads supporting documents → POST /claims/{claimId}/documents (multipart). DocumentService uploads to Mega.nz, saves ClaimDocument with URL.
- User submits → PUT /claims/{claimId}/submit transitions Draft → Submitted. StatusHistory written. ClaimSubmittedEvent + ClaimStatusChangedEvent published.
- Admin views claim in admin dashboard (GET /admin/claims). Admin sets to UnderReview via PUT /admin/claims/{claimId}/review (audit log written via AuditService).
- Admin approves: PUT /admin/claims/{claimId}/approve → Admin Service proxies to Claims Service. Status → Approved. ClaimStatusChangedEvent published.
- OR Admin rejects: PUT /admin/claims/{claimId}/reject → Status → Rejected. ClaimStatusChangedEvent published.
- ClaimStatusChangedEvent consumed by Identity Service → sends email notification to the user.

10. Infrastructure and Configuration

10.1 Docker Compose Services

- rabbitmq: Image rabbitmq:3-management. Ports 5672 (AMQP), 15672 (Management UI). Default credentials: guest/guest. Data persisted in rabbitmq_data volume.
- sqlserver: Image mssql/server:2022-latest. Port 1433. SA password via MSSQL_SA_PASSWORD environment variable. Data persisted in sqlserver_data volume.

10.2 Database Configuration per Service

Service	Connection String Key	DbContext Class	Tables
Identity Service	IdentityConnDb	IdentityDbContext	Users, Passwords, Roles, UserRoles, OtpRecords, ExternalLogins
Policy Service	PolicyConnDb	PolicyDbContext	Policies, InsuranceTypes, InsuranceSubtypes, PolicyDetails, VehicleDetails, HomeDetails, Payments, Discounts
Claims Service	ClaimsConnDb	ClaimsDbContext	Claims, ClaimDocuments, ClaimStatusHistory
Admin Service	AdminConnDb	AdminDbContext	AuditLogs, Reports

10.3 JWT Configuration (Shared Across All Services)

- Jwt:Key — HS256 signing secret. Must be identical across all services.
- Jwt:Issuer — Token issuer identifier (e.g. SmartSure).
- Jwt:Audience — Token audience identifier (e.g. SmartSureUsers).

Identity Service generates tokens. All other services validate them using the same parameters loaded from appsettings.json or environment variables. Tokens contain ClaimTypes.NameIdentifier (UserId), ClaimTypes.Email, and ClaimTypes.Role.

10.4 RabbitMQ / MassTransit Configuration

- All services connect to RabbitMQ on localhost:5672 with credentials guest/guest.
- MassTransit is configured via UsingRabbitMq and ConfigureEndpoints, which auto-creates queues based on consumer class names.
- DotNetEnv is used across all services to load .env files for secrets (connection strings, JWT keys, SMTP credentials).
- Events are C# record types in SmartSure.Shared.Contracts.Events. All consumers implement MassTransit.IConsumer<TEvent>.

10.5 Serilog Logging

- Configured via SerilogExtensions shared helper.
- Enabled in: API Gateway, Claims Service, Admin Service.
- Logs requests, exceptions, and key business events (policy creation, activation, claim transitions).

11. Unit Test Projects

Test Project	Test Classes and Coverage
SmartSure.IdentityService.Tests	UserServiceTests — user CRUD, role assignment, OTP flows
SmartSure.PolicyService.Tests	PolicyMgmtServiceTests — quote, IDV calculation, policy creation, activation, cancellation, discount logic
SmartSure.ClaimsService.Tests	ClaimServiceTests — claim creation, submit, approve, reject, status transitions, conflict guard
SmartSure.AdminService.Tests	AuditServiceTests — audit log creation, paginated retrieval
Frontend (Vitest 4.x)	Angular component and service unit tests configured in tsconfig.spec.json

11.1 Backend Testing Framework

- All four test projects use xUnit and Moq (or NSubstitute) for mocking repositories and services.
- PolicyMgmtServiceTests covers: IDV calculation per depreciation bracket, premium formula, discount calculation (first-time + coupon + capping), policy creation workflow, activation event publishing, cancellation.
- ClaimServiceTests covers: Draft creation, conflict guard, status transitions, approval/rejection, event publishing.
- UserServiceTests covers: Registration, OTP generation/validation, login, role assignment.
- AuditServiceTests covers: Audit log creation and paginated retrieval.

11.2 Frontend Testing Framework

- Vitest 4.x for Angular component and service unit tests.
- Configuration in tsconfig.spec.json.
- Key test files: register.spec.ts, login.spec.ts, home.spec.ts, navbar.spec.ts, app.spec.ts.

12. Security Considerations

- Password Storage: Passwords are hashed using BCrypt (via PasswordHasher helper) and stored in a separate Password table, isolated from the User entity to prevent accidental hash exposure.
- JWT Validation: Each service independently validates JWT tokens using a shared signing key. The API Gateway does NOT validate tokens — this is by design (Ocelot passthrough).
- Refresh Token Storage: Refresh tokens are stored in IMemoryCache (in-process, not distributed). This means they are lost on Identity Service restart. No DB persistence is used for refresh tokens in the current implementation.
- OTP Security: OTPs are 6-digit numeric strings with a 10-minute expiry. A max of 5 verification attempts is enforced before lockout (OtpRecord.Attempts counter). OtpRecord is deleted on successful verification.
- CORS: Restricted to specific origins in the gateway. Credentials are allowed only for those origins.
- Authorization: All sensitive endpoints use [Authorize] attribute. Admin-only endpoints use [Authorize(Roles = "Admin")]. Route guards provide an additional client-side layer.
- File Upload: Documents are uploaded to Mega.nz (third-party encrypted storage). Only the returned URL is stored in ClaimDocument.FileUrl. Content-Type and FileSize are validated on upload.
- Google OAuth: ProviderKey (Google sub ID) is stored in ExternalLogin to uniquely identify OAuth users. No password is stored for Google-auth users (IsGoogleAuth=true).